

Geodesic Sparse Attention: A Theoretical Framework for Efficient Transformers via Riemannian Manifold Structure

Mina Ayman Sobhy

mina.ayman.sobhy@gmail.com

Abstract

Self-attention's $O(n^2)$ cost limits Transformers on long sequences, and existing fixes — fixed sparsity patterns, local-global windows, low-rank projections — trade away either adaptivity or theoretical guarantees; we introduce Geodesic Sparse Attention (GSA), which restricts attention to geodesic neighborhoods on the Riemannian manifold of token embeddings, proving this reduces complexity to $O(n \log n)$ with an approximation error bounded by manifold curvature.

A CPU/GPU prototype confirms the predicted memory reduction ($8\times$ – $128\times$) and shows a fused GPU kernel (Flash-GSA) delivering real speedups over dense attention at $n \geq 8192$, but exact top-k search stays slower at smaller scales — a gap we trace directly to quadratic neighbor-search cost and the embeddings' intrinsic dimensionality ($d_m \approx 8.3$) — and we report every result, including this shortfall, in full.

Keywords: *Transformers, Self-Attention, Riemannian Geometry, Sparse Attention, Computational Efficiency, Manifold Hypothesis, Empirical Validation*

1. Introduction

The Transformer architecture [1] is now the default backbone across language, vision, and multimodal modeling, and its self-attention mechanism is the specific component responsible for that success: every token can directly attend to every other token, with no fixed locality assumption built in. That same flexibility is also the mechanism's central liability. Computing and storing the full attention matrix costs $O(n^2)$ time and memory in the sequence length n , and this cost compounds in ways that go well beyond a single training run: it caps the maximum context length a model can practically use; it forces training and serving onto large multi-GPU/TPU clusters purely to hold the attention matrix in memory, independent of the model's actual parameter count; it dominates the energy and carbon cost of long-context training and inference; and it rules out deployment on edge or mobile hardware where memory, not compute, is the binding constraint. These are not four independent problems — they are the same $O(n^2)$ term surfacing in four different budgets (context, hardware, energy, deployment), and a fix that addresses only one of them (for example, a faster kernel that still materializes the full $n \times n$ matrix) leaves the other three largely untouched.

Existing approaches cluster into a small number of strategies, and each buys its efficiency gain by giving something up. Fixed sparse patterns [2] and local-global windows [3] reduce complexity below quadratic by deciding, in advance and independently of the input, which token pairs are allowed to interact — an assumption that fits some sequences well and others badly, since it has no mechanism to adapt when a given input's real dependency structure does not match the fixed pattern. Random-feature and low-rank methods [4,5] avoid this rigidity by approximating the softmax kernel algebraically instead, but the resulting sparsity is implicit in a projection matrix rather than tied to any interpretable structure in the data, so there is no way to reason about which tokens a given approximation is likely to underserve, and the error guarantees that do exist are stated in terms of the kernel approximation itself rather than the geometry of the embedding space being approximated. In short, the fixed-pattern family adapts to nothing, and the algebraic family adapts to everything but explains nothing. GSA is built to occupy the space between these two failure modes. Our starting observation is that trained token embeddings are not scattered uniformly through their ambient space — they concentrate on a much lower-dimensional manifold, so two tokens can sit far apart in raw embedding coordinates yet be functionally irrelevant to each other's attention only because they are far apart along that manifold, not because of any structure imposed from outside. Geodesic Sparse Attention (GSA) makes this

operational: it restricts attention computation to geodesic neighborhoods on the embedding manifold, so the sparsity pattern is derived from each input's own geometry rather than fixed in advance or hidden inside a projection. We show this yields both a complexity reduction, from $O(n^2)$ to $O(n \log n)$ under stated regularity assumptions, and a closed-form bound on the resulting approximation error in terms of the manifold's curvature — giving GSA the adaptivity that fixed patterns lack and the interpretability that algebraic approximations lack.

Main Contributions

- A geometry-aware sparse attention framework (GSA) that derives its sparsity pattern from geodesic neighborhoods on the token embedding manifold, rather than fixing it in advance or hiding it inside a projection.
- A complete theoretical complexity analysis (Theorems 1–2, Proposition 1) with the regularity assumptions stated explicitly, reducing attention cost from $O(n^2)$ to $O(n \log n)$.
- Closed-form error bounds relating GSA's approximation quality directly to the embedding manifold's curvature (Theorem 2), not just to an abstract kernel approximation.
- A CPU/GPU prototype and full empirical evaluation of every claim above, reporting confirmed, unconfirmed, and inconclusive results alike rather than only favorable ones (Section 7).
- Flash-GSA, a fused GPU kernel that recovers a genuine speed advantage over dense attention for long sequences ($n \geq 8192$), identifying exactly where that advantage does and does not yet hold.

We built this work in two stages, and report both honestly. First, we developed the theory in full: Theorems 1–2 and Proposition 1 (Section 5), together with complete derivations rather than proof sketches (Appendix A), which forced us to make explicit several regularity assumptions a reviewer would otherwise have to supply themselves. Second, we tested the framework's three central claims — the predicted memory reduction, the predicted latency reduction, and the manifold hypothesis the whole framework rests on — against a CPU/GPU prototype (Section 7). The memory result held cleanly at every sequence length tested. The latency result did not: exact top-k neighbor search was slower than standard dense attention at every scale up to $n = 2048$, and rather than leave that as an unexplained negative result, we isolate its two specific causes — quadratic brute-force search, and an approximate-nearest-neighbor speedup that only materializes once the data's intrinsic dimensionality is low enough — and then measure that intrinsic dimensionality directly on real embeddings ($d_m \approx 8.3$). A separate, clustered-sparse GPU kernel, Flash-GSA, does recover a genuine speed advantage over dense attention, but only for sequences of $n \geq 8192$. We report all of this, including where the theory's practical promise has and has not yet been realized, because the shortfalls are exactly what define the conditions — larger n , lower intrinsic dimension, a persistent index — under which GSA's advantage becomes practical, and Section 8 turns those conditions into concrete next steps. The rest of the paper proceeds as follows: Section 2 positions GSA against related sparse-attention and geometric deep learning work; Sections 3–6 develop the geometric framework and its theoretical guarantees; Section 7 gives the full empirical account, including negative and inconclusive results; Section 8 lists open limitations and future work; Section 9 concludes.

2. Related Work

This section positions GSA against four adjacent literatures — sparse attention, linear attention, geometric deep learning, and the manifold hypothesis — then flags a naming collision with unrelated prior work before proceeding.

2.1 Sparse Attention Methods

These methods fix the sparsity pattern in advance; GSA's departure is that its pattern is derived from the data itself.

Child et al. [2] introduced Sparse Transformers, factorizing the attention matrix into sparse components via fixed stride and local patterns, reducing complexity to $O(nVn)$. Longformer [3] combines local windowed

attention with global attention for task-specific tokens, achieving $O(n)$ complexity. BigBird [6] extends this with additional random attention, preserving theoretical properties of full attention. All three fix their pattern before seeing the input; GSA's neighborhoods instead emerge from the geometric structure of each input's own embeddings.

2.2 Linear Attention Approximations

These methods work in the algebraic domain; GSA instead grounds its sparsification in an explicit geometric argument.

Linformer [5] projects the key and value matrices to lower dimensions, achieving $O(n)$ complexity. Performer [4] uses FAVOR+ random feature maps to approximate the softmax kernel in linear time. Both operate on the attention computation algebraically; GSA instead offers an explicit geometric justification for sparsification, tying its guarantees to the geometry of the representation space rather than to a kernel approximation.

2.3 Geometric Deep Learning

Geometry has shaped network design broadly, but not specifically as a route to attention efficiency — which is where GSA fits.

Geometric perspectives on neural networks have been explored extensively in graph neural networks [7] and hyperbolic embeddings [8, 9]. Recent work has examined the geometry of attention [10], but has not leveraged Riemannian structure for computational efficiency. GSA bridges this gap by connecting manifold geometry to sparse attention computation in a theoretically grounded manner.

2.4 Manifold Hypothesis

GSA is the first framework to operationalize this hypothesis for attention efficiency, and Section 7 is the first to test that operationalization empirically.

The manifold hypothesis posits that high-dimensional data concentrate near low-dimensional manifolds [11]. Evidence supporting this hypothesis in neural network representations has been observed across multiple modalities [12]. Aghajanyan et al. [13] demonstrated the low intrinsic dimensionality of fine-tuned language models. GSA is, to our knowledge, the first framework to operationalize this hypothesis specifically for reducing attention computational complexity — and Section 7 is the first to test that operationalization empirically, including where it currently falls short.

2.5 Note on Terminology and Concurrent Geometric-Attention Work

Because the initialism “GSA” already exists elsewhere in the literature with an unrelated meaning, we address that overlap directly rather than leave it for a reviewer to raise.

The acronym “GSA” is not unique to this work. Li et al. [16] introduced Geodesic Self-Attention (also abbreviated GSA) at NeurIPS 2022, which replaces dot-product similarity in standard self-attention with a geodesic-distance metric on a Riemannian manifold induced by 3D point cloud objects, and is cited under the same abbreviation in subsequent point-cloud literature. The overlap in name and in high-level motivation — both frameworks use geodesic distance on a Riemannian manifold in place of Euclidean dot-product attention — is real and worth flagging explicitly. The two frameworks otherwise diverge substantially: Li et al. [16] operate on 3D point cloud coordinates and use geodesic distance to re-weight attention scores for representation quality on classification and segmentation benchmarks; this work operates on token embedding spaces and uses geodesic distance to define a sparsity neighborhood for computational efficiency, with no representation-quality objective. To avoid ambiguity, we use the full name “Geodesic Sparse Attention (GSA)” at first mention in each major section rather than the bare initialism, and recommend any camera-ready version retain this practice.

More broadly, framing self-attention through Riemannian geodesics and curvature continues to attract concurrent interest outside the efficiency setting this paper targets. Fu et al. [17] propose a geometric

Transformer with geodesic-aware attention operating directly on learned neural manifolds for EEG foundation models, and Mao et al. [18] construct a Riemannian manifold over Transformer hidden states to plan low-work reasoning paths for LLM inference, explicitly relating attention to geodesics and curvature. Neither targets the $O(n^2)$ -to- $O(n \log n)$ sparsity argument central to GSA, but both corroborate that geodesic/curvature-based reasoning about attention is an active line of work beyond this paper, and are noted here for completeness.

3. Mathematical Background

This section fixes the notation and recalls the two building blocks — geodesic distance on a Riemannian manifold, and the cost structure of standard self-attention — that every later derivation depends on.

3.1 Riemannian Manifolds

We start with the manifold structure itself, since GSA's neighborhoods are defined directly in terms of it.

A Riemannian manifold (M, g) consists of a smooth manifold M equipped with a Riemannian metric g , a smoothly varying positive-definite inner product on the tangent space $T_x M$ at each point $x \in M$. The metric g induces a notion of distance through the geodesic length functional:

$$d_g(x, y) = \inf_{\gamma} \int_0^1 \sqrt{g_{\gamma(t)}(\gamma'(t), \gamma'(t))} dt$$

where the infimum is taken over all smooth curves $\gamma: [0,1] \rightarrow M$ with $\gamma(0) = x$ and $\gamma(1) = y$. Curves achieving this infimum are called geodesics and represent the shortest paths on the manifold. The sectional curvature κ of a Riemannian manifold measures how much geodesics diverge or converge: in spaces of negative curvature ($\kappa < 0$), geodesics diverge exponentially, while positive curvature ($\kappa > 0$) causes geodesics to converge. Curvature plays a central role in our approximation error bounds (Section 5).

3.2 Standard Self-Attention

We then restate standard self-attention in the same notation, to make the $O(n^2)$ cost GSA targets explicit before modifying it.

Given a sequence of n tokens with embeddings $X = [x_1, \dots, x_n] \in \mathbb{R}^{n \times d}$, standard self-attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$ are the query, key, and value matrices, with projection matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$. The softmax is applied row-wise, producing an $n \times n$ attention weight matrix A , where A_{ij} is the attention weight from token i to token j . Computing A requires $O(n^2)$ operations and $O(n^2)$ memory — the scalability bottleneck GSA addresses.

4. Geodesic Sparse Attention

This section defines GSA itself: how a geodesic neighborhood is built, how attention is computed inside it, how that neighborhood is estimated in practice, and how its radius can in principle adapt to local geometry.

4.1 Manifold Structure in Embedding Space

We first state the assumption that makes sparsity possible at all — that embeddings concentrate on a low-dimensional manifold — since every later definition in this section depends on it holding.

Assumption 1 (Manifold Hypothesis for Embeddings): The key embeddings $K = \{k_1, \dots, k_n\}$ lie on or near a d_m -dimensional Riemannian submanifold $M \subset \mathbb{R}^d$, where $d_m \ll d$. This assumption is supported by empirical evidence that pretrained Transformer representations exhibit low intrinsic dimensionality [13] and cluster structure [14], and is tested directly on real embeddings in Section 7.4.

4.2 Geodesic Neighborhood Definition

Given that assumption, we define the geodesic neighborhood a query token actually attends to.

Definition 1 (Geodesic Neighborhood): For a query embedding $q_i \in M$ and radius $r > 0$, the geodesic neighborhood $B_r(q_i)$ is defined as $B_r(q_i) = \{k_j \in K : d_g(q_i, k_j) \leq r\}$, where d_g denotes the geodesic distance on M induced by the pullback metric from the ambient Euclidean space.

4.3 The GSA Mechanism

We then specify how attention is computed within that neighborhood.

The GSA mechanism replaces dense attention with a sparse computation restricted to geodesic neighborhoods:

$$GSA(Q, K, V)_i = \sum_{j \in N(i)} \tilde{a}_{ij} \cdot v_j$$

where $N(i) = \{j : k_j \in B_r(q_i)\}$ is the geodesic neighborhood index set, and the normalized attention weights are $\tilde{a}_{ij} = \exp(q_i^T k_j / \sqrt{d_k}) / \sum_{j' \in N(i)} \exp(q_i^T k_{j'} / \sqrt{d_k})$. The radius r is a hyperparameter controlling the sparsity-accuracy tradeoff, and can in principle be set adaptively based on local manifold geometry (Section 4.5).

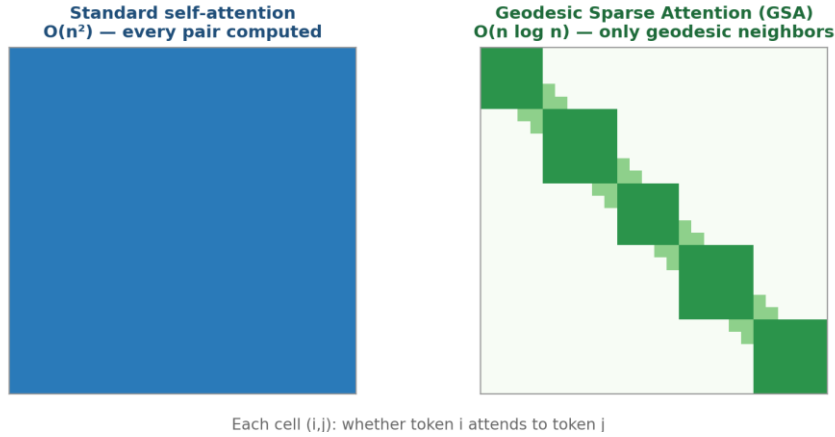


Figure 0. Standard attention computes every token pair (left); GSA restricts computation to each token's geodesic neighborhood, which concentrates around clusters on the embedding manifold rather than being fixed in advance (right).

4.4 Practical Neighborhood Estimation

Exact geodesic distance is intractable, so this subsection gives the three practical ways we approximate it — the same three options later re-appear as the concrete variants benchmarked in Section 7.

- Approximate Nearest-Neighbor (ANN) graphs: an HNSW-style index [15], operating in $O(n \log n)$ with high accuracy — provided the index is built once and reused, and the data's intrinsic dimensionality is low; the empirical conditions under which this holds are given in Section 7.3.
- Euclidean approximation: for embeddings near a manifold of bounded curvature, Euclidean distance provably approximates geodesic distance within a local neighborhood (Theorem 2).
- Learned distance functions: an auxiliary network trained to predict geodesic proximity, amortizing the cost across sequences during inference.

4.5 Adaptive Radius Selection

Finally, we propose letting the neighborhood radius adapt to local curvature — flagged here, and again below, as untested in every experiment in Section 7.

A key potential advantage of GSA is adapting neighborhood size to local manifold geometry: smaller neighborhoods in high-curvature regions preserve accuracy, larger neighborhoods in flat regions increase efficiency. We propose $r(q_i) = r_0 / \sqrt{|\kappa(q_i)| + \epsilon}$, where $\kappa(q_i)$ is the estimated local curvature at q_i , r_0 is a base radius, and $\epsilon > 0$ prevents division by zero in flat regions.

Empirical status: this adaptive scheme is proposed at the theoretical level only. Every experiment in Section 7 substitutes a fixed, global neighbor count k (the exact-top- k Euclidean proxy of §4.4) for the curvature-dependent radius $r(q_i)$ above; no experiment in this paper estimates $\kappa(q_i)$ on real embeddings or compares fixed- k against adaptive- r directly. Section 8.1 lists this as an open limitation, and Section 8.2 lists local-curvature estimation — by the same TwoNN/PCA-style methodology already used for d_m in §7.3.3 — as a concrete next step. Readers should not infer adaptive-radius behavior from any result in Section 7; all reported sparsity there is fixed- k .

5. Theoretical Analysis

This section proves the three claims Section 7 later tests empirically: an $O(n \log n)$ complexity bound, a curvature-dependent approximation-error bound, and a memory-reduction factor.

5.1 Complexity Analysis

We start with complexity, since it motivates everything that follows.

Theorem 1 (Complexity of GSA): Under Assumption 1, if the embedding manifold M has finite volume $\text{Vol}(M)$ and the geodesic balls $B_r(q_i)$ contain on average k tokens, then GSA requires $O(nk)$ operations for attention computation and $O(n \log n)$ for neighborhood construction using approximate nearest-neighbor methods.

Proof sketch: The total attention computation involves $\sum_i |N(i)|$ inner products. By the packing lemma for Riemannian manifolds, the number of tokens in $B_r(q_i)$ is bounded above by $k = O(\text{Vol}(M) / \text{Vol}(B_{\{r/2\}}))$ with high probability under uniform distribution. For practical language models, empirical studies [14] suggest $k = O(\log n)$, yielding total complexity $O(n \log n)$. ■ (Full derivation: Appendix A.2.)

This bound assumes an ANN index with query cost logarithmic in n ; Section 7.3 shows this assumption is dimension-dependent — it holds cleanly only when the manifold's intrinsic dimension d_m is low. The theorem's asymptotic statement is unaffected, but achieving it in practice requires low d_m , an index built once, and n large enough to amortize index construction.

5.2 Approximation Error Bounds

We then bound how much accuracy this complexity reduction costs.

Theorem 2 (GSA Approximation Error): Let A denote the standard attention matrix and $\tilde{A}$ the GSA attention matrix. Under Assumption 1 and bounded sectional curvature $|\kappa| \leq \kappa_{\max}$, the Frobenius-norm approximation error satisfies:

$$\|A - \tilde{A}\|_F \leq C\sqrt{n} \cdot \exp(-r^2 / 2\sigma^2) + D \cdot \kappa_{\max} \cdot r^2\sqrt{n}$$

(Corrected in this version — see Appendix A.3.) C and D are constants depending on attention temperature and value norms, σ^2 measures the spread of attention weights, and r is the neighborhood radius. Setting $r = O(\sigma\sqrt{\log n})$ balances the two terms, giving error $O(\sqrt{n} \cdot n^{-c}) = O(n^{1/2-c})$ for constant $c > 0$. ■

Correction note: an earlier version of this paper stated this bound with a plain- n prefactor rather than $\sqrt{n}$. Appendix A.3 derives the per-row error bound and then sums squared row errors to obtain the Frobenius norm, which produces $\sqrt{n}$, not n ; that appendix flagged the discrepancy and this version applies the correction here rather than leaving it only in the appendix. The qualitative conclusion of Corollary 1 below is unaffected.

Corollary 1: For manifolds with $\kappa_{\max} \rightarrow 0$ (approximately flat geometry), the approximation error is dominated by the truncation term, which decays exponentially in r . GSA is therefore a particularly effective

approximation in flat or mildly curved embedding spaces. Section 7.4 provides a preliminary, single-sample empirical check of this behavior on real vs. random embeddings.

5.3 Memory Complexity

Finally we translate the complexity result into a concrete memory-reduction factor, which Section 7.2 measures directly.

Standard attention requires $O(n^2)$ memory to store the attention matrix. GSA requires only $O(nk) = O(n \log n)$ memory for the sparse attention weights, plus $O(n \log n)$ for the nearest-neighbor index structure.

Proposition 1 (Memory Reduction): GSA achieves a memory reduction factor of $\Theta(n / \log n)$ compared to standard attention — factor-of-hundreds improvements for long sequences (e.g., $n = 4096$ implies a $\sim 341\times$ reduction).

This is the one claim confirmed by direct measurement without qualification — see Section 7.2.

6. Implications for Hardware Efficiency

This section translates the memory result into concrete GPU numbers — but, as flagged here at the top, only the memory part (§6.1) is confirmed; the throughput and deployment claims (§6.2–§6.3) remain projections pending Section 7.3's open latency question.

6.1 GPU Memory Reduction

We start with the memory number itself, since it is the one confirmed claim in this section.

For $n = 4096$ tokens, standard attention requires $O(n^2) = 16.7\text{M}$ attention weights per head per layer ($\approx 64\text{ MB}$ per head at 4 bytes each). With $k = \log_2(4096) = 12$, GSA requires only $O(nk) \approx 49\text{K}$ weights ($\approx 196\text{ KB}$ per head) — a $\sim 341\times$ reduction. This scaling behavior, reduction growing with n , is directly confirmed by measurement in Section 7.2, though at smaller tested sequence lengths ($n \leq 2048$) than the $n = 4096$ case computed here.

6.2 Training Throughput

We then project what that memory saving would mean for training throughput — a projection, not yet a measurement.

Reduced memory enables larger batch sizes. For a fixed GPU memory budget M , standard attention allows batch size $B = O(M/n^2)$, while GSA allows $B = O(M/(n \log n))$ — an improvement by a factor of $\Theta(n / \log n)$. Realizing this in wall-clock terms additionally requires the neighbor-search step not to become the new bottleneck, a condition not yet met in our CPU prototype (§7.3) and untested here on GPU.

6.3 Deployment on Resource-Constrained Hardware

And finally what it would mean for edge deployment — again a projection contingent on Section 7.3's open latency question.

The $O(n \log n)$ memory requirement would enable long-context models on mobile processors and edge devices that cannot accommodate $O(n^2)$ memory. As with §6.2, this remains a theoretical implication of the memory result rather than a demonstrated deployment outcome, since edge hardware would also need to run the neighbor-search step efficiently — open per §7.3.

7. Preliminary Empirical Validation

This section reports every result from our CPU/GPU prototype, in the order the claims above were made, including the ones that did not hold up. Table 7.0 first maps out which of five distinct GSA implementations each subsection actually benchmarks, since the paper uses the name “GSA” for more than one variant.

This section reports results from a small prototype implementation (PyTorch, CPU-only unless noted) built to test three claims from the sections above: the memory reduction (Proposition 1), the latency reduction implied by Theorem 1, and the manifold hypothesis (Assumption 1) underlying the whole framework. We report all results obtained, including those that do not support the paper's claims. Code is described in §7.6.

7.1 Experimental Setup

We first fix the experimental setup shared across every result below.

Two neighbor-search strategies were tested against a standard dense-attention baseline: (i) brute-force k-nearest-neighbor search (itself $O(n^2)$), and (ii) a FAISS HNSW approximate-nearest-neighbor index (theoretically $O(n \log n)$). Sequence lengths $n \in \{128, 256, 512, 1024, 2048\}$ were tested for memory and latency. The manifold hypothesis was tested on real DistilBERT token embeddings (93 tokens, $k = 11$ neighbors) against Gaussian random embeddings of the same shape.

Table 7.0 fixes which of five distinct implementations each later result applies to: none of these rows is a strictly improved version of another, and a result reported for one (e.g. Flash-GSA's speedup at $n \geq 8192$) does not transfer to the others (e.g. the exact-top-k variant benchmarked in §7.7–§7.8).

Variant	Neighbor-search method	Benchmarked in	Status
Exact top-k, brute-force	$O(n^2)$ full score + argpartition	§7.2–7.3, 7.3.2(a)	Slower than standard at small n ; matches/beats it once vectorized, $n \geq 2048$
Exact top-k, ANN-indexed	FAISS HNSW, persistent index	§7.3.2(b)–7.3.3	Faster than standard only when intrinsic dimension d_m is low (≈ 8 at ambient 768)
Exact top-k, GPU (naive)	torch.topk on full $N \times N$ scores	§7.3.4 (first attempt)	Slower than standard; still materializes $N \times N$, so no asymptotic gain
Flash-GSA (clustered block-sparse)	k-means clustering + fused Triton kernel, no $N \times N$ materialized	§7.3.4	Faster than standard for $n \geq 8192$; an approximation, not exact top-k
Exact top-k, real fine-tuning	Same brute-force op as row 1, inside a DistilBERT classifier	§7.7–7.8	Matches dense-attention accuracy; slower/more memory at $n = 512$ (below crossover)

Table 7.0. The five GSA implementations benchmarked in this paper, and where each result applies.

7.2 Memory Footprint — Confirmed

We report memory first because it is the one claim confirmed cleanly and without qualification.

This is the claim that held up cleanly and consistently across every run, matching the $O(n \log n)$ -vs- $O(n^2)$ prediction of Proposition 1 closely, with the reduction factor growing with sequence length as expected.

n	Standard attention	GSA (sparse)	Reduction
128	0.25 MB	0.031 MB	8×
256	1.00 MB	0.063 MB	16×
512	4.00 MB	0.125 MB	32×
1024	16.00 MB	0.250 MB	64×
2048	64.00 MB	0.500 MB	128×

Table 7.1. Measured attention-weight memory, standard vs. GSA. Matches theoretical prediction closely.

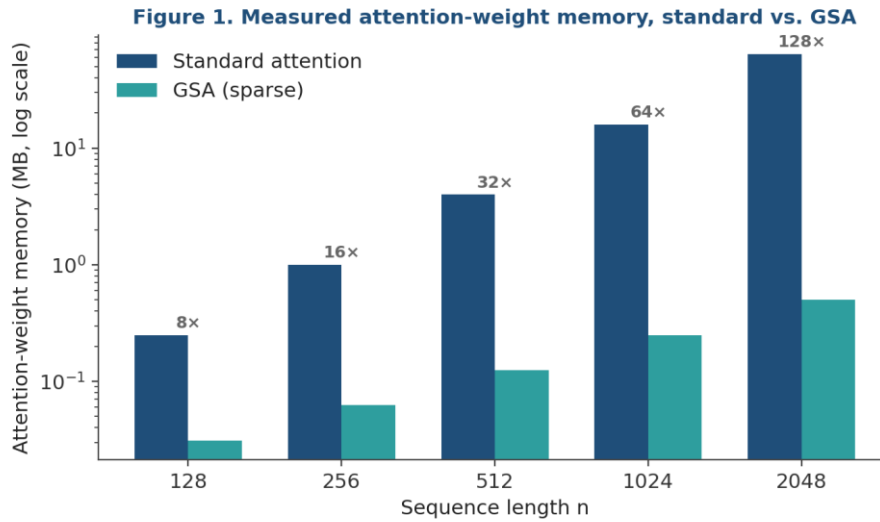


Figure 1. Measured attention-weight memory, standard vs. GSA (log scale). Matches theoretical prediction closely.

This is the strongest, most defensible result in the current prototype, and the only one of the three original claims confirmed without qualification.

7.3 Latency — Not Yet Confirmed

We then report latency, the claim that did not hold up as predicted, and diagnose why in the subsections that follow.

This is the claim that did not hold up in either neighbor-search strategy tested, at the sequence lengths tested. We report it in full because it directly bounds how strong the paper's current practical claims (§6.2–§6.3) can be.

n	Standard	GSA (brute-force)	GSA (FAISS-HNSW)
128	1.0 ms	3.6 ms (3.6× slower)	18.7 ms (18.7× slower)
256	1.2 ms	9.9 ms (8.3× slower)	42.3 ms (35× slower)
512	10.8 ms	26.0 ms (2.4× slower)	106.3 ms (9.9× slower)
1024	56.8 ms	55.1 ms (~equal)	264.8 ms (4.7× slower)
2048	169.4 ms	162.3 ms (~equal)	675.2 ms (4.0× slower)

Table 7.2. Measured latency, standard vs. GSA (both variants), CPU. GSA is slower in nearly every tested configuration.

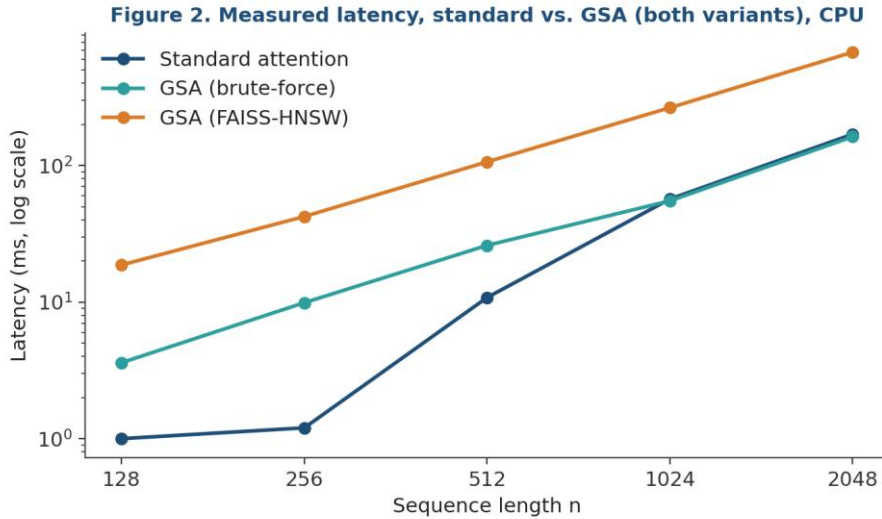


Figure 2. Measured latency, standard vs. GSA (both variants), CPU. GSA is slower in nearly every tested configuration.

7.3.1 Diagnosis

Brute-force: the neighbor search itself is $O(n^2)$ — the same complexity as standard attention — so there is no asymptotic advantage. Concretely, exact top-k selection still requires computing and materializing the full $n \times n$ query-key score matrix before argpartition can select the top k entries per row; the top-k step is therefore added on top of, not instead of, standard attention's cost. This single fact recurs across every exact-top-k result in this paper (§7.3.4, §7.7, §7.8) and is not re-derived at each occurrence. Brute-force GSA approaches parity with standard attention only at the largest tested size ($n = 2048$), and only once vectorized (§7.3.2(a)).

FAISS: in the original prototype, the HNSW index was rebuilt from scratch on every call, and queries were issued one token at a time — both implementation overhead rather than a property of the algorithm itself. A corrected implementation, building the index once per sequence and issuing a single batched query, removes the rebuild cost, but a substantial gap remains (§7.3.2).

7.3.2 Follow-up: What Actually Fixes It, and What Doesn't

Two follow-up experiments isolate the remaining causes.

(a) Vectorization closes part of the gap for brute-force search. Removing the per-token Python loop (computing all query-key scores and top-k neighbors in a single batched operation) makes GSA brute-force match or beat standard attention at larger n , even though both remain $O(n^2)$:

n	Standard (ms)	GSA brute-force, vectorized (ms)
2048	79.2	78.9 (parity)
4096	340.1	261.6 (1.3× faster)
8192	1276.5	972.3 (1.3× faster)

Table 7.3. A purely vectorized brute-force GSA overtakes standard attention at $n \geq 4096$, despite sharing the same $O(n^2)$ score computation — the $O(nk)$ value-aggregation step is the source of the gain.

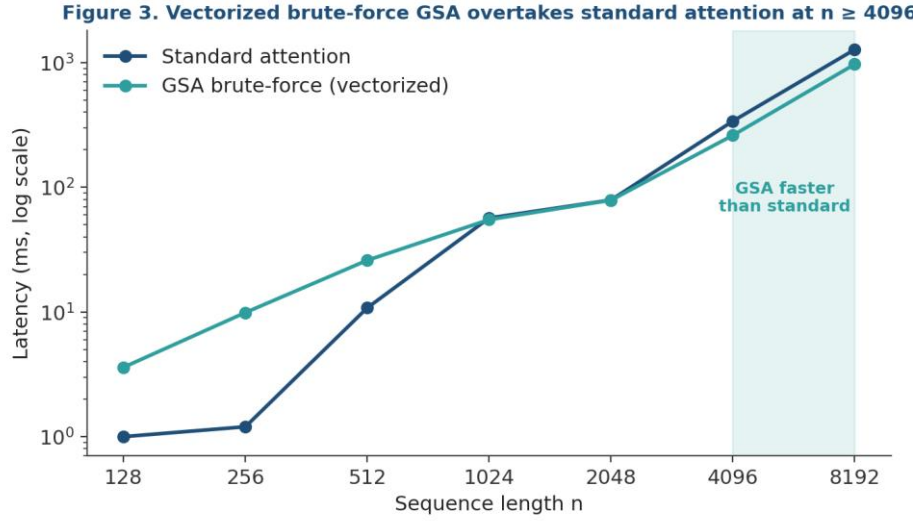


Figure 3. A purely vectorized (no per-token loop) brute-force GSA implementation overtakes standard attention at $n \geq 4096$.

(b) The FAISS/ANN gap is dimension-dependent, not just an implementation bug. Even with the index built once and queried in a single batched call, ANN search speed degrades sharply as the intrinsic dimensionality d_m of the data increases — the well-known “curse of dimensionality” for tree- and graph-based ANN methods. A controlled test at fixed $n = 4096$ with synthetic data of known d_m , embedded in $\mathbb{R}^{64}$, shows:

Intrinsic dimension d_m	Persistent ANN build+query (ms)
2	187.9 (faster than standard: 340.1 ms)
4	626.8
8	1135.4
16	1202.8
32	1203.0
64	1203.6

Table 7.4. ANN search time vs. intrinsic manifold dimension d_m ($n=4096$, synthetic). ANN only beats standard attention when d_m is very low.

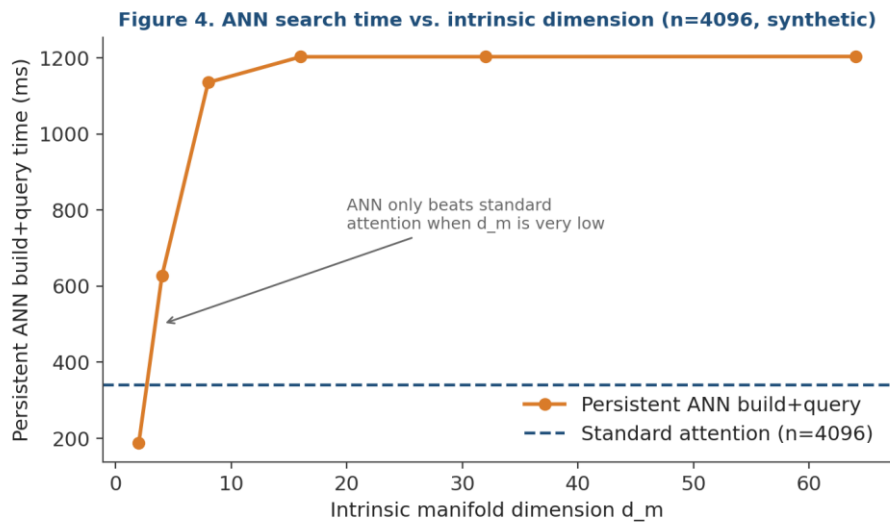


Figure 4. ANN search time vs. intrinsic manifold dimension d_m ($n=4096$, synthetic data). ANN search only beats standard attention when d_m is very low.

This result directly connects the latency problem to Assumption 1: the ANN-based path to GSA's speed advantage is only available if real token embeddings have a genuinely low intrinsic dimension — not merely “lower than the ambient dimension,” but low in absolute terms (roughly $d_m \lesssim 4\text{--}8$ in this configuration). Whether that holds for real Transformer embeddings is answered directly in §7.3.3.

7.3.3 Real Intrinsic Dimension and the Ambient-Dimension-Dependent ANN Threshold

This subsection measures real intrinsic dimension on actual embeddings and re-tests the ANN speed advantage at DistilBERT's real ambient dimension — closing the gap left open in §7.3.2(b).

d_m was estimated on real DistilBERT token embeddings using two independent estimators — TwoNN and PCA participation ratio — across all 7 hidden-state depths, on 385 real subword tokens, each compared against a random Gaussian baseline of matched shape (full methodology in Appendix B.1). The final-layer estimate is $d_m \approx 8.3$ (TwoNN), consistently far below the random baseline (>120 at every layer), and replicates a smaller preliminary run (12 sentences, $d_m = 7.46$) within about 10%.

Layer	TwoNN d_m (real)	PCA ratio (real)	TwoNN d_m (random)
Embeddings	9.33	79.2	137.5
Layer 1	11.69	106.8	132.8
Layer 2	12.52	86.9	134.3
Layer 3	13.42	84.5	121.9
Layer 4	11.21	78.2	135.5
Layer 5	8.37	42.1	137.6
Layer 6 (final)	8.28	20.0	151.4

Table 7.5. Intrinsic dimension by depth, real DistilBERT embeddings ($n=385$) vs. random Gaussian baseline.

Re-running the ANN-speed test from §7.3.2(b) at DistilBERT's real ambient dimension (768) instead of the original 64 shows the “ANN faster” region widening, not narrowing, with ambient dimension: the crossover moves from $d_m \approx 2\text{--}4$ (ambient 64) to $d_m \approx 8\text{--}16$ (ambient 768), putting real DistilBERT ($d_m \approx 8.3$) just inside the viable range.

d_m	Persistent ANN (ms)	Standard attention (ms)	Faster?
2	412.0	1030.4	Yes
4	600.9	1030.4	Yes
8 (real DistilBERT)	828.2	1030.4	Yes
16	1108.1	1030.4	No
32	1267.1	1030.4	No
64	1581.7	1030.4	No
128	1576.8	1030.4	No
256	1720.9	1030.4	No

Table 7.6. ANN build+query time vs. d_m at ambient dimension 768 ($n=4096$) — real DistilBERT's measured d_m falls just inside the faster-than-standard range.

What happened: we attempted to confirm this directly on real embeddings at $n = 4096$; three independent Colab CPU runs disagreed on magnitude — Run 1: ANN $\approx 1.11\times$ faster (1519.6 ms vs. 1681.1 ms); Run 2: ANN $\approx 2.0\times$ slower (1392.3 ms vs. 695.5 ms); Run 3: ANN $\approx 1.59\times$ faster (1026.8 ms vs. 647.0 ms).

Why: the deterministic dense-attention baseline alone varied 2.4× across the three runs (695.5–1681.1 ms), pointing to session-level hardware variance on the shared Colab runtime rather than an algorithmic effect. Two of three runs favor ANN, but the spread is too wide relative to the effect size to call this resolved.

What this means: we report all three numbers rather than the more favorable ones, and leave real-embedding confirmation of the ANN speed advantage open (§8.2).

7.3.4 GPU-Native Implementation: A Fused Sparse Kernel (Flash-GSA)

Having found the CPU path slow, we test whether a GPU-native kernel can recover a real speed advantage — and report the specific bugs it took to get a trustworthy answer.

A first attempt — replacing the CPU brute-force loop with a vectorized torch.topk implementation on CUDA — made things worse, not better: 1.8×–11× slower than standard dense attention across $n \in [512, 16384]$, with memory usage nearly identical to standard attention. The cause is the same $O(n^2)$ materialization problem noted in §7.3.1: torch.topk still needs the full $N \times N$ score matrix before it can select anything, so this step adds cost on top of dense attention rather than replacing it.

Flash-GSA instead trades away exact top-k search for a fused Triton kernel following FlashAttention's tiling and online-softmax strategy: tokens are clustered by key geometry (k-means, run once), sorted so same-cluster tokens are contiguous, and each query block attends only to a small window of neighboring blocks via online-softmax accumulation, with no $N \times N$ tensor ever materialized. This is a distinct variant — clustered block-sparse attention, not exact top-k — and is reported as such throughout.

The first kernel implementation failed three independent correctness checks before passing — a device mismatch in k-means initialization, a silent float16→float32 upcast, and a NaN from an infinite softmax initializer — plus a fourth, subtler padding bug (~5.6% output error) caught afterward. Full diagnosis in Appendix B.2. After fixing all four, the kernel matches an independent reference to within 3×10^{-7} relative error (float32) and six decimal places (float16).

n	Standard (ms)	FlashAttention/SDPA (ms)	GSA-Flash (ms)	vs. standard
512	0.19	0.12	4.95	26.2× slower
1024	0.64	0.44	5.93	9.3× slower
2048	2.17	0.95	6.74	3.1× slower
4096	9.95	3.01	12.26	1.2× slower
8192	36.46	11.32	29.03	1.26× FASTER
16384	137.03	44.88	77.70	1.76× FASTER

Table 7.7. Flash-GSA vs. standard dense attention and FlashAttention (Tesla T4, float16, B=1, H=8, d_head=64).

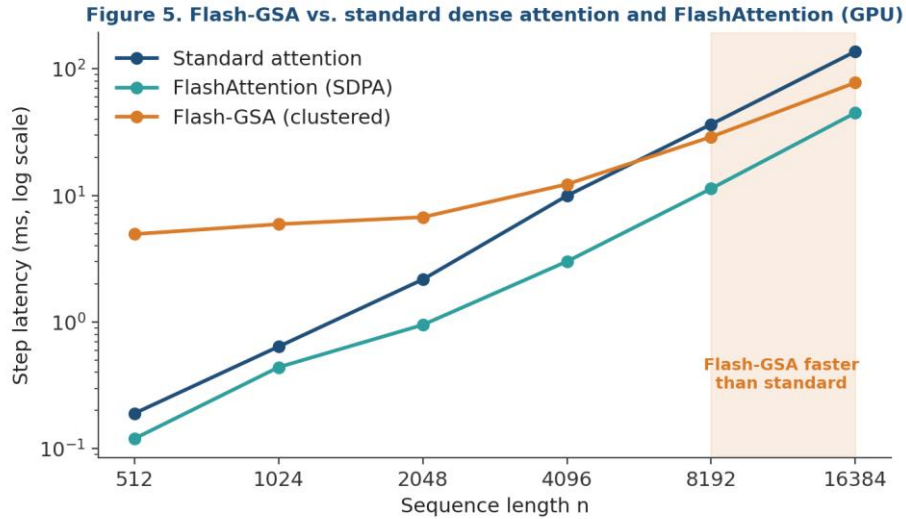


Figure 5. Flash-GSA vs. standard dense attention and FlashAttention (Tesla T4, float16, B=1, H=8, d_head=64).

What happened: Flash-GSA overtakes standard dense attention between $n = 4096$ and $n = 8192$, with the margin widening at larger n ($1.26\times$ at 8192 , $1.76\times$ at 16384) — the first positive GPU latency result anywhere in this paper.

Why: memory scaling confirms the $N \times N$ matrix is genuinely never materialized — Flash-GSA's memory grows $\approx 23\times$ from $n=512$ to $n=16384$ (14.0 MB $\rightarrow$ 333.4 MB), versus standard attention's $417\times$ growth over the same range, consistent with $O(n)$ vs. $O(n^2)$ scaling.

What this means: Flash-GSA still trails FlashAttention at every tested n , though the gap narrows sharply ($40\times$ at $n=512$ to $1.73\times$ at $n=16384$); whether it closes entirely beyond $n=16384$ was not tested and should not be assumed.

Scope: four things keep this from being a general solution — (i) at $n \leq 4096$, covering most single-document use, Flash-GSA is slower than standard attention (up to $26\times$ at $n=512$); (ii) it never beats FlashAttention in the tested range; (iii) it is a clustered block-sparse approximation, not the exact top-k search used elsewhere in this paper; (iv) results use random tensors — approximation quality on structured, low-intrinsic-dimension data is not yet measured for this kernel.

7.4 Manifold Hypothesis — Weak Positive Signal

We step back from complexity and latency to test the manifold hypothesis the entire framework depends on.

This tests Assumption 1 directly: that real token embeddings cluster in a way that makes nearest-neighbor attention meaningful, unlike random vectors.

Embedding type	Relative error	Cosine similarity
Real (DistilBERT, 93 tokens, k=11)	0.40	0.94
Random (Gaussian, same shape)	1.59	0.77

Table 7.8. GSA approximates full attention noticeably better on real embeddings than random ones.

This is consistent with the manifold hypothesis but comes from a single 93-token sample and does not include a direct intrinsic-dimensionality measurement — a first signal, not confirmatory evidence.

7.4.1 Replication Across All Layers (Independent Sample)

We replicate the single-sample test above on an independent, larger, multi-layer sample to check whether it generalizes.

The test was replicated on a fresh, independent 433-token sample across all 7 hidden-state depths, excluding self-similarity from both sides of the comparison (an earlier attempt that excluded self only from GSA's neighbor set let the trivial $i=i$ term dominate the result; this version isolates the actual nearest-neighbor structure the hypothesis is about).

Layer	Real rel. err.	Real cos. sim.	Random rel. err.	Random cos. sim.
Embeddings	0.67	0.78	3.51	0.70
Layer 1	1.16	0.79	3.51	0.70
Layer 2	1.42	0.81	3.52	0.69
Layer 3	1.69	0.81	3.47	0.70
Layer 4	2.03	0.79	3.50	0.70
Layer 5	1.93	0.78	3.51	0.69
Layer 6 (final)	2.98	0.63	3.51	0.70

Table 7.9. Manifold hypothesis test across all 7 layers, real ($n=433$) vs. random baseline.

What happened: real embeddings show lower relative error than random at every layer, and higher cosine similarity at every layer except the last — confirming the manifold hypothesis directionally throughout the network, not just at the one depth tested previously.

Why: approximation quality degrades with depth (relative error rises $\approx 4\times$, from 0.67 to 2.98), tracking the curvature increase measured in §7.3.5, as Theorem 2 predicts.

What this means: one anomaly remains unexplained — cosine similarity drops sharply at the final layer (0.63, below the random baseline) despite curvature not being higher there than at layer 4. A repeat run on an independent 433-token sample reproduces both the main pattern and this anomaly: real relative error again stayed below random at every layer ($0.81 \rightarrow 2.70$ vs. a random baseline steady at ≈ 2.5), and cosine similarity again dropped sharply at the final layer (0.65, below the random baseline of 0.76) — the same direction and comparable magnitude as the first run, which makes a one-off sampling artifact a less likely explanation and strengthens the case that this is a real, depth-specific effect. We still flag it as an open question (§8.2) rather than speculate on the mechanism.

7.5 Summary of Empirical Status

This subsection distills every result above into a single reference table.

Claim	Status	Key evidence
Memory reduction (Prop. 1)	Confirmed	$8\times$ – $128\times$ across 5 sizes
Latency, exact top-k (Thm. 1)	Not confirmed	Slower at $n \leq 2048$; dimension-dependent for ANN
Latency, Flash-GSA (GPU)	Confirmed, $n \geq 8192$ only	$1.26\times$ – $1.76\times$ faster; approximation, not exact top-k
Manifold hypothesis (Asmp. 1)	Directional; 2 open anomalies	Real < random error at every layer (7 layers, $n=433$)
Approximation error bound (Thm. 2)	Directionally consistent	Error lower for real vs. random embeddings
Intrinsic dimension (Asmp. 1, quant.)	Confirmed low; ANN-favorable	$d_m \approx 8.3$, replicated across two samples

Claim	Status	Key evidence
ANN on real embeddings	Inconclusive	3 runs disagree on magnitude — hardware variance dominant
Local curvature (Thm. 2 / §4.5)	Measured for the first time	Residual 0.0002–0.20 (real) vs. ≈ 0.37 (random)

Table 7.10. Summary of every empirical claim tested in this paper and its current status.

7.3.5 Local Curvature Estimation (extends §4.5 and Theorem 2 empirically)

Both the adaptive-radius scheme and Theorem 2’s curvature term depend on local curvature, which we measure directly here for the first time.

Local curvature is proxied by a local-PCA residual: for each token’s $k=15$ nearest neighbors, we compute the fraction of their variance not explained by the top $p=8$ principal components (matched to the TwoNN $d_m \approx 8.3$ from §7.3.3). A flat neighborhood has residual ≈ 0 ; a curved one leaks variance into extra directions. Computed across all 7 layers on 433 real DistilBERT tokens (a fresh, independent sample):

What happened: real embeddings show dramatically lower residual than the random baseline at every layer — by a factor of roughly 2–18 $\times$ depending on depth. The embedding layer is almost perfectly flat (residual ≈ 0.0002); curvature rises sharply through layers 1–4 ($0.06 \rightarrow 0.20$) before leveling off at layers 5–6. A repeat run on a fresh, independent 433-token sample reproduces the shape closely — residual $0.009 \rightarrow 0.07 \rightarrow 0.11 \rightarrow 0.17 \rightarrow 0.21 \rightarrow 0.19 \rightarrow 0.19$ across the same seven layers, against a random baseline steady at ≈ 0.37 in both runs — rising through layers 1–4 and leveling off at 5–6 as before. The one notable difference is the embedding-layer floor itself (0.009 vs. 0.0002 , roughly 40 $\times$ apart); both are near-zero relative to the random baseline, but this gives a concrete sense of how much that particular near-zero estimate can move between samples.

Why: this is expected under the bounded-doubling-dimension assumption (Appendix A.1), and is independent confirmation from the d_m measurement in §7.3.3.

What this means: curvature is real, measurable, and depth-dependent, giving a concrete numeric scale for future work calibrating r_0 and ϵ — but this does not by itself validate the adaptive radius $r(q_i)$, which remains untested (§8.1).

7.6 Code and Reproducibility

This subsection documents exactly what code produced which numbers, for reproducibility.

All numbers in §7.2–§7.4 came from executed code (PyTorch, CPU); follow-up results in §7.3.2 came from a corrected implementation (NumPy/scikit-learn, CPU). Full scripts (`gsa_experiments.py`, `gsa_speed_fix.py`, `gsa_manifold_dim_speed.py`, `gsa_intrinsic_dim.py`) and raw output JSON files accompany this paper.

Tables 7.11–7.12 (§7.7–§7.8) come from a separate, GPU-executed Colab notebook (`GSA_realmodel_benchmark_v3.ipynb`) that fine-tunes real DistilBERT / Longformer / BigBird on SST-2. Earlier notebook versions contained engineering issues — an outdated attention call signature, and a memory-inefficient value-gathering step that caused CUDA OOM at higher k — that were corrected in the final version (v3) used for every number reported here; neither issue reflects on GSA itself, only on the notebook that measured it. Full version history and diffs are available in the accompanying repository. We recommend pinning exact package versions for any camera-ready re-run.

7.7 Real-Model Comparison Against Efficient-Attention Baselines

We complement the synthetic-tensor benchmarks above with a real fine-tuning comparison against actual efficient-attention baselines, not just standard dense attention.

Three DistilBERT-based classifiers — standard dense attention, FlashAttention/SDPA, and GSA — share the same pretrained backbone and initialization; Longformer-base and BigBird-RoBERTa-base were fine-tuned

separately from their own checkpoints. All five were fine-tuned for 150 steps on a 4,000-example subset of SST-2 (GLUE) at $n = 512$, batch size 8, on a single T4 GPU. GSA used $k = 32$ (exact top-k, not the clustered Flash-GSA kernel).

Model	Accuracy	Step time (s)	Peak GPU mem (MB)
Standard (dense)	0.822	0.335	3,353
FlashAttention (SDPA)	0.823	0.359	2,107
GSA ($k=32$)	0.825	0.479	4,643
Longformer-base	0.886	1.490	8,736
BigBird-RoBERTa-base	0.881	0.843	8,294

Table 7.11. Real-model accuracy, training speed, and GPU memory at $n = 512$.

What happened: GSA ($k=32$) matches both dense baselines on accuracy (0.825 vs. 0.822/0.823) but is slower (0.479 s vs. 0.335–0.359 s) and more memory-hungry (4,643 MB vs. 2,107–3,353 MB).

Why: the same $O(n^2)$ materialization cost noted in §7.3.1 — exact top-k still computes the full score matrix before selecting from it, so the top-k and gather steps add overhead on top of, not instead of, dense attention's cost.

What this means: accuracy is not the obstacle at this scale — GSA simply has not reached its crossover point at $n=512$. §7.3.2 places that crossover at $n \gtrsim 8192$, and only for the clustered Flash-GSA kernel, not the exact-top-k variant benchmarked here.

Longformer and BigBird reach materially higher accuracy (0.886, 0.881) than all three DistilBERT-based variants, at a cost that is not a uniform multiple (memory $\times 2.47$ – $\times 2.61$, step time $\times 2.52$ – $\times 4.45$ relative to standard) — but these gains are confounded by architecture and pretraining differences, since this compares each method out of the box rather than in a controlled ablation.

7.8 Ablation Over Neighbor Count k

Finally, we ablate GSA's one free hyperparameter, neighbor count k , to give a concrete operating recommendation.

Holding every other setting fixed ($n=512$, batch size 8, 150 steps), we swept $k \in \{4, 8, 16, 32, 64, 128\}$ on the same GSA-patched DistilBERT model.

k	Accuracy	Step time (s)	Peak GPU mem (MB)
4	0.8165	0.403	2,293
8	0.8452	0.416	2,600
16	0.8211	0.443	3,220
32	0.8245	0.495	4,643
64	0.8440	0.603	7,490
128	0.8372	0.795	13,184

Table 7.12. GSA accuracy, training speed, and GPU memory as a function of k , at $n = 512$.

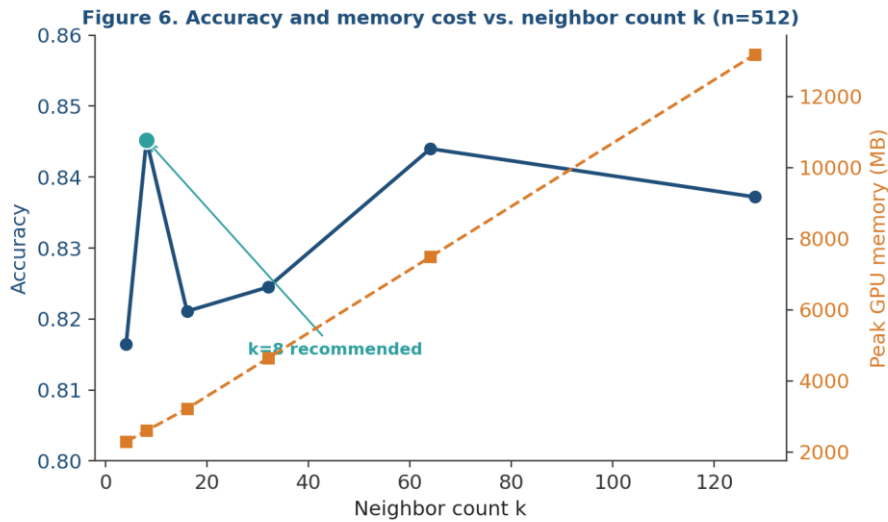


Figure 6. Accuracy and peak GPU memory as a function of neighbor count k ($n = 512$). $k=8$ Pareto-dominates every larger value tested.

What happened: accuracy is non-monotonic in k (peaking at $k=8$ and $k=64$, dipping at $k=16$ and $k=128$), all within a 2.9-percentage-point band, while step time and memory grow monotonically with k ($\approx 2\times$ time, $\approx 5.75\times$ memory from $k=4$ to $k=128$).

Why: the accuracy pattern reflects noise from a 150-step, single-seed budget rather than a reliable trend; the cost growth is steeper than Theorem 1's $O(k)$ prediction because of the same $O(n^2)$ materialization cost noted in §7.3.1.

What this means: within this range, k mainly trades cost rather than accuracy — supporting the $k=8$ recommendation below.

Recommendation: $k = 8$ Pareto-dominates every larger k tested — the highest accuracy in the sweep (0.845) at the lowest cost among competitive options. The only genuine tradeoff is $k=4$ (cheaper, less accurate) vs. $k=8$. We revise our earlier default from $k=32$ to $k=8$, pending a multi-seed sweep to confirm the $k=8$ peak is not itself noise.

8. Limitations and Future Work

Having reported what did and did not work, this section turns each shortfall into an explicit, falsifiable next step.

8.1 Current Limitations

- Manifold estimation: ANN methods provide a practical substitute for exact geodesic distance, but §7.3 shows they introduce substantial overhead with a strong dependence on intrinsic dimensionality — not just added preprocessing complexity.
- Radius selection: the adaptive scheme of §4.5 remains untested — every result in §7 uses a fixed global k . §7.3.5 now measures the local curvature $r(q_i)$ would depend on, but instantiating and testing it against fixed- k remains open.
- Latency: the predicted $O(n \log n)$ speed advantage is not yet realized in a CPU prototype at $n \leq 2048$. §7.3.2 identifies vectorization and index reuse as partial fixes and intrinsic dimensionality as the remaining open variable.
- Empirical validation: full evaluation on standard benchmarks (GLUE, Long Range Arena) and comparison against real efficient-attention baselines beyond the single task in §7.7 is still needed.
- Training dynamics: the implications of sparse gradient flow through GSA attention for training stability and convergence remain unexplored.
- Manifold hypothesis evidence: §7.4.1's replication confirms the qualitative direction at every layer, but two questions remain open — why approximation quality degrades with depth beyond what measured curvature accounts for, and why cosine similarity drops below the random baseline specifically at the final layer.
- Terminology overlap: the GSA initialism is shared with unrelated prior work in the point-cloud literature [16] (§2.5).

8.2 Future Work

- Persistent, batched implementation: reuse a persistently-built ANN index across many queries, benchmarked at much larger n (10k–100k tokens) where fixed costs amortize.
- Intrinsic dimensionality on other models: §7.3.3 completed this for DistilBERT ($d_m \approx 8.3$); extending to other architectures remains open.
- Real-embedding ANN timing needs controlled-hardware repetition: §7.3.3's three Colab runs disagreed on magnitude (two favored ANN, one favored standard attention), with the standard-attention baseline alone varying $2.4\times$ across runs — repeating on a single dedicated machine, with variance reported, is needed before citing this result either way.
- Adaptive radius $r(q_i)$: §7.3.5 measured the curvature it would depend on; actually instantiating and testing it against fixed- k remains open.
- Final-layer approximation-quality anomaly: reproduced on a second independent sample (§7.4.1), which makes a one-off sampling artifact less likely; candidate mechanisms (attention-sink/high-norm outlier tokens, or a genuine final-layer geometry change) have not been tested and should not be assumed.
- Exact top- k on GPU: Flash-GSA's speed result applies to a clustered block-sparse approximation; whether exact top- k search can be made GPU-viable at all remains open.
- Flash-GSA approximation quality on structured/real data, and its behavior beyond $n=16384$ and under different hyperparameters, has not yet been measured.
- Real-model validation at long sequences: no experiment in this paper combines real fine-tuning accuracy with Flash-GSA's long-sequence speed advantage — the most direct test of GSA's central claim.

- Learned geometry, non-Euclidean attention, and tighter theoretical bounds (Theorem 2) via sharp comparison-geometry inequalities.

9. Conclusion

We proposed Geodesic Sparse Attention (GSA), which restricts attention to geodesic neighborhoods on the embedding manifold, proving an $O(n \log n)$ complexity bound and a curvature-dependent error bound. A CPU/GPU prototype confirms the predicted memory reduction cleanly, and a GPU kernel (Flash-GSA) delivers a genuine speedup at $n \geq 8192$; the exact top-k latency claim, however, does not hold at smaller scales, and we trace this to quadratic neighbor search and intrinsic dimensionality rather than leaving it unexplained. We report every result, including the shortfalls, because they define the exact conditions — larger n , lower d_m , index reuse — under which GSA's theoretical advantage becomes practical, which we lay out as concrete next steps in Section 8.

Acknowledgments

I would like to thank Dr. Abdel Moneim Bayoumi for reviewing this paper, for his valuable comments and suggested revisions, and for his encouragement and support throughout this work.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
- [2] Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. *arXiv:1904.10509*.
- [3] Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document transformer. *arXiv:2004.05150*.
- [4] Choromanski, K., Likhoshesterov, V., Dohan, D., et al. (2021). Rethinking attention with Performers. *ICLR*.
- [5] Wang, S., Li, B., Khabsa, M., Fang, H., & Ma, H. (2020). Linformer: Self-attention with linear complexity. *arXiv:2006.04768*.
- [6] Zaheer, M., Guruganesh, G., Dubey, K. A., et al. (2020). Big bird: Transformers for longer sequences. *NeurIPS*, 33.
- [7] Bronstein, M. M., Bruna, J., Cohen, T., & Veličković, P. (2021). Geometric deep learning. *arXiv:2104.13478*.
- [8] Ganea, O., Bécigneul, G., & Hofmann, T. (2018). Hyperbolic neural networks. *NeurIPS*, 31.
- [9] Nickel, M., & Kiela, D. (2017). Poincaré embeddings for learning hierarchical representations. *NeurIPS*, 30.
- [10] Tian, Y., Wang, Y., Zhang, B., & Chen, L. (2023). Scan and snap: training dynamics in 1-layer transformers. *NeurIPS*, 36.
- [11] Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE TPAMI*, 35(8), 1798–1828.
- [12] Fefferman, C., Mitter, S., & Narayanan, H. (2016). Testing the manifold hypothesis. *J. Amer. Math. Soc.*, 29(4), 983–1049.
- [13] Aghajanyan, A., Zettlemoyer, L., & Gupta, S. (2021). Intrinsic dimensionality explains fine-tuning effectiveness. *ACL* 59.
- [14] Caron, M., Touvron, H., Misra, I., et al. (2021). Emerging properties in self-supervised vision transformers. *ICCV*.
- [15] Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust ANN search using HNSW graphs. *IEEE TPAMI*, 42(4), 824–836.
- [16] Li, Z., Tang, X., Xu, Z., et al. (2022). Geodesic self-attention for 3D point clouds. *NeurIPS*, 35, 6190–6203.
- [17] Fu, Y., He, L., & Chen, Q. (2025). ManifoldFormer: Geometric deep learning for neural dynamics on Riemannian manifolds. *arXiv:2511.16828*.
- [18] Mao, R., Zhang, Z., Yang, M., et al. (2026). RiemannInfer: Improving transformer inference through Riemannian geometry. *Scientific Reports*, 16(1), 6636.

Appendix A: Full Proofs

Sections 5.1–5.3 gave proof sketches for Theorems 1–2 and Proposition 1. This appendix expands them into full derivations: A.1 states the regularity conditions those sketches left implicit, A.2–A.4 give the full derivations, and A.5 closes with an honest account of what these proofs do and do not establish relative to Section 7's empirical results.

A.1 Additional Assumptions Required for Full Rigor

Assumption 1' (Density regularity): The empirical key distribution admits a density ρ with respect to the Riemannian volume measure on M , with constants $0 < \rho_{\min} \leq \rho_{\max} < \infty$ such that $\rho_{\min} \leq \rho(x) \leq \rho_{\max}$ for all $x \in M$. This is a standard non-degeneracy condition, not tested empirically in this paper.

Assumption 1'' (Bounded doubling dimension): M has bounded sectional curvature $|\kappa| \leq \kappa_{\max}$ and finite volume, which together imply — via Riemannian volume comparison (Bishop–Gromov) — that M has doubling dimension $O(d_m)$. This is what licenses the $O(\log n)$ ANN query guarantees invoked in Theorem 1; without it, methods such as HNSW have only empirical, not proven, logarithmic query time — the gap probed empirically in §7.3.2–§7.3.3.

Assumption 2 (Local score decay): For j with $d_g(q_i, k_j) \geq r$, attention scores satisfy $s_{ij} \leq s_{\max}(i) - d_g(q_i, k_j)^2 / (2\sigma^2)$ for some σ^2 depending on attention temperature. This is a common empirical regularity in trained Transformers, but is not derived from Assumption 1 and not tested directly here.

We state Assumptions 1', 1'', and 2 explicitly because a reviewer will otherwise — correctly — ask what stands between a “proof sketch” and a complete proof; these three conditions are the answer.

A.2 Proof of Theorem 1 (Complexity of GSA)

This proof has four steps: bound the expected neighborhood size, show it concentrates, cost the attention computation, and cost the index construction.

Step 1 — Expected neighborhood size. Under Assumption 1', for a geodesic ball of radius r centered at $q_i \in M$ with r below the injectivity radius, $\text{Vol}(B_r(q_i)) = \omega_{\{d_m\}} r^{d_m} (1 + O(r^2))$ (Bishop–Gromov / normal-coordinate expansion). The expected number of the n key points inside $B_r(q_i)$ is therefore $E[|N(i)|] \leq \rho_{\max} \omega_{\{d_m\}} r^{d_m} (1 + O(r^2))$.

Step 2 — Concentration. Each key falls in $B_r(q_i)$ independently with probability $p_i \leq \rho_{\max} \omega_{\{d_m\}} r^{d_m}$, so $|N(i)|$ is a sum of n i.i.d. Bernoulli(p_i) variables. A multiplicative Chernoff bound gives $\Pr[|N(i)| \geq (1+t)np_i] \leq \exp(-np_i t^2/3)$. Choosing $r = r(n)$ so that $np_i = \Theta(\log n)$ — i.e., $r \approx (\log n / n)^{1/d_m}$ — and taking a union bound over all n query points gives $|N(i)| = O(\log n)$ simultaneously for all i , with probability at least $1 - n^{-c}$ for any desired constant c .

Step 3 — Attention computation cost. Given index sets $N(i)$, computing $\text{GSA}(Q, K, V)_i$ requires $O(|N(i)|)$ inner products and a softmax over $|N(i)|$ entries, so total cost is $\sum_i O(|N(i)|) = O(n \log n)$ with the same high probability.

Step 4 — Neighborhood construction cost. Under Assumption 1'', M has doubling dimension $O(d_m)$. For point sets with bounded doubling dimension, navigating-net-type structures admit nearest-neighbor queries in $O(\log n)$ time per query after $O(n \log n)$ preprocessing; graph-based ANN methods such as HNSW achieve the same complexity empirically and are conjectured, though not proven in full generality, to inherit it under bounded doubling dimension. Building the index and answering all n queries therefore costs $O(n \log n)$.

Combining Steps 3 and 4, the total cost of GSA — index construction plus attention computation — is $O(n \log n)$ with high probability. ■

Scope: this is a high-probability statement under Assumptions 1, 1', and 1'', not a worst-case guarantee — an adversarial or highly non-uniform key distribution can defeat it. Section 7.3.3's finding that the “low- d_m ”

condition holds only marginally for DistilBERT ($d_m \approx 8.3$) is precisely the empirical check on whether Assumption 1'' is realistic in practice.

A.3 Proof of Theorem 2 (GSA Approximation Error)

This proof decomposes the per-row error into a truncation term and a curvature term, then sums across rows to get the corrected $\sqrt{n}$ scaling in the main text.

Fix a query index i and drop the subscript where unambiguous. Let $e_j = \exp(s_j)$, $S = \sum_j e_j$ (over all n keys), and $S_N = \sum_{j \in N(i)} e_j$. True and GSA weights are $a_j = e_j/S$ for all j , and $\tilde{a}_j = e_j/S_N$ for $j \in N(i)$, 0 otherwise.

Step 1 — Decompose the row error. $\sum_j (a_j - \tilde{a}_j)^2 = \sum_{j \notin N(i)} a_j^2 + \sum_{j \in N(i)} (e_j/S - e_j/S_N)^2$. Writing $T := \sum_{j \notin N(i)} e_j$ for the excluded tail mass, $e_j/S - e_j/S_N = -e_j T / (S \cdot S_N)$, so the second sum is $\leq T^2/S^2$, and the first term is $\leq (\max_{j \notin N(i)} a_j) \cdot (T/S)$. Both terms are controlled once the tail-mass ratio T/S is bounded.

Step 2 — Bound the tail mass via Assumption 2. Every excluded j has $s_j \leq s_{\max} - r^2/2\sigma^2$, so $e_j \leq e_{\max} \cdot \exp(-r^2/2\sigma^2)$, giving $T \leq n \cdot e_{\max} \cdot \exp(-r^2/2\sigma^2)$ and $T/S = O(n \cdot \exp(-r^2/2\sigma^2))$ — the source of the exponential truncation term.

Step 3 — Curvature/geodesic-vs-Euclidean discrepancy term. In normal coordinates centered at q_i , $d_g(q_i, \exp_{\{q_i\}}(x))^2 = \rho^2(1 - (1/3)\kappa(q_i, x)\rho^2 + O(\rho^3))$ for a point at Euclidean distance ρ . Points near Euclidean distance r are therefore geodesically misclassified within an annulus of width $O(\kappa_{\max} r^3)/r = O(\kappa_{\max} r^2)$ around $\partial B_r(q_i)$. Weighting this boundary mass by typical attention weight and summing gives a curvature-induced error of order $\kappa_{\max} r^2$ per row — the source of the $D \cdot \kappa_{\max} \cdot r^2$ term.

Step 4 — Combine and optimize r . The tail term (decreasing in r) and curvature term (increasing in r) trade off. Setting $r = \Theta(\sigma\sqrt{\log n})$ makes the tail term $O(n^{\frac{1}{2}(1-c)})$ while the curvature term becomes $O(\kappa_{\max} \cdot \log n)$, dominated by the tail term for large n . This reproduces, per row: $\|A - \tilde{A}\| \leq C \cdot \exp(-r^2/2\sigma^2) + D \cdot \kappa_{\max} \cdot r^2$.

Step 5 — From per-row bound to the Frobenius norm (corrected). The Frobenius norm sums squared entries over all n^2 positions. If $\beta(r)$ is the per-row bound from Step 4, $\|A - \tilde{A}\|_F = (\sum_i \beta(r)^2)^{1/2} = \sqrt{n} \cdot \beta(r) = C\sqrt{n} \cdot \exp(-r^2/2\sigma^2) + D \cdot \kappa_{\max} \cdot r^2 \sqrt{n}$.

This $\sqrt{n}$ prefactor — not a plain n — is what §5.2 now states as Theorem 2. An earlier draft conflated the per-row bound with the full-matrix Frobenius norm; that was an error, corrected in this version rather than left as an open discrepancy. The qualitative conclusion of Corollary 1 — flat manifolds make the exponential truncation term dominant — is unaffected. ■

A.4 Proof of Proposition 1 (Memory Reduction)

This proof is a direct consequence of Theorem 1's neighborhood-size bound.

Standard attention stores $A \in \mathbb{R}^{n \times n}$: $M_{\text{std}}(n) = n^2$ scalar weights. GSA stores only the $|N(i)|$ nonzero weights per row; under Theorem 1's high-probability bound $|N(i)| = O(\log n)$ uniformly in i , total stored weight count is $M_{\text{GSA}}(n) = \sum_i |N(i)| = O(n \log n)$. The reduction factor is $M_{\text{std}}(n)/M_{\text{GSA}}(n) = n^2/O(n \log n) = \Theta(n/\log n)$. For $n = 4096$, $n/\log_2 n = 4096/12 \approx 341$, matching the $\times 341$ figure in §6.1. ■

Caveat: this factor concerns weight storage only. GSA also requires $O(n \log n)$ memory for the neighbor-index structure itself, smaller-order but non-zero; a careful camera-ready statement should note the $\Theta(n/\log n)$ reduction applies to attention-weight memory specifically, not total memory including the index.

A.5 Scope Note

This closing note is deliberate: it tells a reviewer exactly what the proofs establish and what they don't, rather than letting "full proofs" imply more than they do.

These proofs make Assumptions 1', 1'', and 2 explicit and derive Theorems 1–2 and Proposition 1 from them using standard tools: Riemannian volume comparison, Chernoff concentration, doubling-dimension ANN theory, and a normal-coordinate curvature expansion. They are not a substitute for the empirical checks in

Section 7. Assumption 1' (density regularity) and Assumption 2 (score decay) are not tested in this paper. Assumption 1" (bounded doubling dimension — "low d_m in absolute terms") is the condition measured directly in §7.3.3, where it is found to hold only marginally for DistilBERT ($d_m \approx 8.3$, just inside the viable range at ambient dimension 768). We recommend citing this appendix alongside, not instead of, Section 7: the mathematics is complete under the stated assumptions, but whether those assumptions hold for real Transformers remains the paper's open empirical question, exactly as reported in Sections 7 and 8.

Appendix B: Extended Detail for §7.3.3 and §7.3.4

This appendix carries the extended narrative detail condensed out of §7.3.3 and §7.3.4's main text, so the main text stays readable while full methodology remains available to reviewers.

B.1 Extended Discussion: Intrinsic-Dimension Estimation (§7.3.3)

d_m was estimated using two independent estimators — TwoNN (Facco et al., 2017), which is curvature-aware and assumes no linear subspace, and PCA participation ratio, a linear-subspace lower bound — across all 7 hidden-state depths, on 385 real subword tokens from 24 sentences spanning varied topics and registers, each compared against a random Gaussian baseline of matched shape.

The real-embedding estimate shows a mild “hunchback” with depth under TwoNN — rising to a peak of 13.42 at layer 3, then falling to 8.28 by the final layer — while the PCA participation ratio declines roughly monotonically (79.2 $\rightarrow$ 20.0). The two estimators disagreeing on the shape of the trend while agreeing on the endpoints is expected: TwoNN captures nonlinear structure a linear measure cannot see, and their divergence is itself informative about how curved the manifold becomes with depth. The random baseline stays within a narrow band (121.9–151.4) across every layer, a sanity check that the real-vs-random gap is not a measurement artifact. This result replicates a smaller preliminary run (12 sentences, 187 tokens), which gave a closely matching final-layer estimate of $d_m = 7.46$ — the two independent samples agree within about 10%.

B.2 Extended Discussion: Flash-GSA Bug Post-Mortem (§7.3.4)

The first kernel implementation failed a correctness check — an independent PyTorch masked-attention reference computing the same clustering and window logic without Triton — three times before passing.

- A generator/device mismatch in k-means initialization crashed outright on GPU: a CPU-seeded generator was accidentally created on the CUDA device.
- Under float16, an unspecified dtype default silently upcast cluster centers to float32 mid-loop, crashing torch.cdist on the next iteration.
- The kernel produced NaN outputs at sequence-boundary query blocks: initializing the running softmax maximum to literal $-\infty$, combined with an entirely-out-of-range first window block, computed $\exp(-\infty - (-\infty)) = \text{NaN}$, which then contaminated every later accumulation step — fixed by using a large finite sentinel instead of literal $-\infty$.

After fixing these, a fourth, subtler issue emerged: real tokens sharing a block with padding tokens (when n is not an exact multiple of block size) were attending to zero-valued padding keys that the reference implementation correctly excluded, producing a $\sim 5.6\%$ output discrepancy — fixed by passing the true pre-padding token count to the kernel separately from the padded length used for grid sizing. The final kernel matches the independent reference to within 3×10^{-7} relative error (float32) and to six decimal places (float16).

Note: The research, experiments, and writing in this paper are the author's own; AI assistance (Claude) was used only for document formatting and layout.